第 13 章　深度生成模型

> 我不能创造的东西，我就不了解.
>
> ——理查德·菲利普斯·费曼（Richard Phillips Feynman）
> 1965 年诺贝尔物理奖获得者

概率生成模型（Probabilistic Generative Model），简称生成模型，是概率统计和机器学习领域的一类重要模型，指一系列用于随机生成可观测数据的模型. 假设在一个连续或离散的高维空间 $\mathcal{X}$ 中，存在一个随机向量 $\boldsymbol{X}$ 服从一个未知的数据分布 $p_r(\boldsymbol{x}), \boldsymbol{x} \in \mathcal{X}$. 生成模型是根据一些可观测的样本 $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \cdots, \boldsymbol{x}^{(N)}$ 来学习一个参数化的模型 $p_\theta(\boldsymbol{x})$ 来近似未知分布 $p_r(\boldsymbol{x})$，并可以用这个模型来生成一些样本，使得“生成”的样本和“真实”的样本尽可能地相似. 生成模型通常包含两个基本功能：概率密度估计和生成样本（即采样）. 图13.1以手写体数字图像为例给出了生成模型的两个功能示例，其中左图表示手写体数字图像的真实分布 $p_r(\boldsymbol{x})$ 以及从中采样的一些“真实”样本，右图表示估计出了分布 $p_\theta(\boldsymbol{x})$ 以及从中采样的“生成”样本.

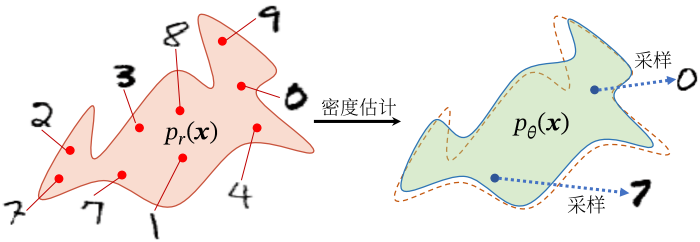


图 13.1　生成模型的两个功能

生成模型的应用十分广泛，可以用来建模不同的数据，比如图像、文本、声音等. 但对于一个高维空间中的复杂分布，密度估计和生成样本通常都不容易实现. 一是高维随机向量一般比较难以直接建模，需要通过一些条件独立性来简化模型，二是给定一个已建模的复杂分布，也缺乏有效的采样方法.

深度生成模型就是利用深度神经网络可以近似任意函数的能力来建模一个复杂分布 $p_r(\boldsymbol{x})$ 或直接生成符合分布 $p_r(\boldsymbol{x})$ 的样本. 本章先介绍概率生成模型的基本概念,然后介绍两种深度生成模型:变分自编码器和生成对抗网络.

13.1 概率生成模型

生成模型一般具有两个基本功能:密度估计和生成样本.

概率密度估计简称密度估计,参见第9.2节.

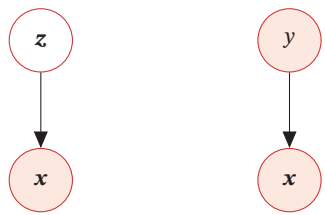
13.1.1 密度估计

给定一组数据 $\mathcal{D} = \{\boldsymbol{x}^{(n)}\}_{n=1}^{N}$,假设它们都是独立地从相同的概率密度函数为 $p_r(\boldsymbol{x})$ 的未知分布中产生的. 密度估计(Density Estimation)是根据数据集 $\mathcal{D}$ 来估计其概率密度函数 $p_\theta(\boldsymbol{x})$.

在机器学习中,密度估计是一类无监督学习问题. 比如在手写体数字图像的密度估计问题中,我们将图像表示为一个随机向量 $\boldsymbol{X}$,其中每一维都表示一个像素值. 假设手写体数字图像都服从一个未知的分布 $p_r(\boldsymbol{x})$,希望通过一些观测样本来估计其分布. 但是,手写体数字图像中不同像素之间存在复杂的依赖关系(比如相邻像素的颜色一般是相似的),很难用一个明确的图模型来描述其依赖关系,所以直接建模 $p_r(\boldsymbol{x})$ 比较困难. 因此,我们通常通过引入隐变量 $\boldsymbol{z}$ 来简化模型,这样密度估计问题可以转换为估计变量 $(\boldsymbol{x}, \boldsymbol{z})$ 的两个局部条件概率 $p_\theta(\boldsymbol{z})$ 和 $p_\theta(\boldsymbol{x}|\boldsymbol{z})$. 一般为了简化模型,假设隐变量 $\boldsymbol{z}$ 的先验分布为标准高斯分布 $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. 隐变量 $\boldsymbol{z}$ 的每一维之间都是独立的. 在这个假设下,先验分布 $p(\boldsymbol{z};\theta)$ 中没有参数. 因此,密度估计的重点是估计条件分布 $p(\boldsymbol{x}|\boldsymbol{z};\theta)$.

如果要建模含隐变量的分布(如图13.2a),就需要利用 EM 算法来进行密度估计. 而在 EM 算法中,需要估计条件分布 $p(\boldsymbol{x}|\boldsymbol{z};\theta)$ 以及近似后验分布 $p(\boldsymbol{z}|\boldsymbol{x};\theta)$. 当这两个分布比较复杂时,我们可以利用神经网络来进行建模,这就是变分自编码器的思想.

EM 算法参见第11.2.2.1节.



(a) 含隐变量的生成模型 (b) 带标签的生成模型

图 13.2 生成模型

13.1.2 生成样本

生成样本就是给定一个概率密度函数为 $p_\theta(\boldsymbol{x})$ 的分布，生成一些服从这个分布的样本，也称为采样. 我们在第11.5节中介绍了一些常用的采样方法.

采样方法参见第11.5节.

对于图13.2a中的图模型，在得到两个变量的局部条件概率 $p_\theta(\boldsymbol{z})$ 和 $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ 之后，我们就可以生成数据 $\boldsymbol{x}$，具体过程可以分为两步进行：

（1） 根据隐变量的先验分布 $p_\theta(\boldsymbol{z})$ 进行采样，得到样本 $\boldsymbol{z}$.

（2） 根据条件分布 $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ 进行采样，得到样本 $\boldsymbol{x}$.

为了便于采样，通常 $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ 不能太过复杂. 因此，另一种生成样本的思想是从一个简单分布 $p(\boldsymbol{z}), \boldsymbol{z} \in \mathcal{Z}$（比如标准正态分布）中采集一个样本 $\boldsymbol{z}$，并利用一个深度神经网络 $g: \mathcal{Z} \to \mathcal{X}$ 使得 $g(\boldsymbol{z})$ 服从 $p_r(\boldsymbol{x})$. 这样，我们就可以避免密度估计问题，并有效降低生成样本的难度，这正是生成对抗网络的思想.

13.1.3 应用于监督学习

除了生成样本外，生成模型也可以应用于监督学习. 监督学习的目标是建模样本 $\boldsymbol{x}$ 和输出标签 y 之间的条件概率分布 $p(y|\boldsymbol{x})$. 根据贝叶斯公式，

$$p(y|\boldsymbol{x}) = \frac{p(\boldsymbol{x}, y)}{\sum_y p(\boldsymbol{x}, y)}. \tag{13.1}$$

我们可以将监督学习问题转换为联合概率分布 $p(\boldsymbol{x}, y)$ 的密度估计问题.

图13.2b给出了带标签的生成模型的图模型表示，可以用于监督学习. 在监督学习中，比较典型的生成模型有朴素贝叶斯分类器、隐马尔可夫模型.

参见第11.1.2节.

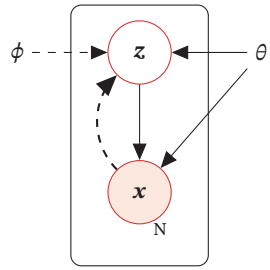
判别模型 和生成模型相对应的另一类监督学习模型是判别模型（Discriminative Model）. 判别模型直接建模条件概率分布 $p(y|\boldsymbol{x})$，并不建模其联合概率分布 $p(\boldsymbol{x}, y)$. 常见的判别模型有Logistic 回归、支持向量机、神经网络等. 由生成模型可以得到判别模型，但由判别模型得不到生成模型.

13.2 变分自编码器

13.2.1 含隐变量的生成模型

假设一个生成模型（如图13.3所示）中包含隐变量，即有部分变量是不可观测的，其中观测变量 $\boldsymbol{X}$ 是一个高维空间 $\mathcal{X}$ 中的随机向量，隐变量 $\boldsymbol{Z}$ 是一个相对低维的空间 $\mathcal{Z}$ 中的随机向量.

本章中，我们假设 $\boldsymbol{X}$ 和 $\boldsymbol{Z}$ 都是连续随机向量.



实线表示生成模型，虚线表示变分近似.

图 13.3 变分自编码器

这个生成模型的联合概率密度函数可以分解为

$$p(\boldsymbol{x}, \boldsymbol{z}; \theta) = p(\boldsymbol{x}|\boldsymbol{z}; \theta)p(\boldsymbol{z}; \theta), \tag{13.2}$$

其中 $p(\boldsymbol{z}; \theta)$ 为隐变量 $\boldsymbol{z}$ 先验分布的概率密度函数，$p(\boldsymbol{x}|\boldsymbol{z}; \theta)$ 为已知 $\boldsymbol{z}$ 时观测变量 $\boldsymbol{x}$ 的条件概率密度函数，θ 表示两个密度函数的参数. 一般情况下，我们可以假设 $p(\boldsymbol{z}; \theta)$ 和 $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$ 为某种参数化的分布族，比如正态分布. 这些分布的形式已知，只是参数 θ 未知，可以通过最大化似然来进行估计.

给定一个样本 $\boldsymbol{x}$，其对数边际似然 $\log p(\boldsymbol{x}; \theta)$ 可以分解为

$$\log p(\boldsymbol{x}; \theta) = ELBO(q, \boldsymbol{x}; \theta, \phi) + \mathrm{KL}(q(\boldsymbol{z}; \phi), p(\boldsymbol{z}|\boldsymbol{x}; \theta)), \tag{13.3}$$

其中 $q(\boldsymbol{z}; \phi)$ 是额外引入的变分密度函数，其参数为 ϕ，$ELBO(q, \boldsymbol{x}; \theta, \phi)$ 为证据下界，

参见公式(11.49).

$$ELBO(q, \boldsymbol{x}; \theta, \phi) = \mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z};\phi)}\left[\log \frac{p(\boldsymbol{x}, \boldsymbol{z}; \theta)}{q(\boldsymbol{z}; \phi)}\right]. \tag{13.4}$$

最大化对数边际似然 $\log p(\boldsymbol{x}; \theta)$ 可以用 EM 算法来求解. 在 EM 算法的每次迭代中，具体可以分为两步：

EM 算法参见第11.2.2.1节.

（1） E 步：固定 θ，寻找一个密度函数 $q(\boldsymbol{z}; \phi)$ 使其等于或接近于后验密度函数 $p(\boldsymbol{z}|\boldsymbol{x}; \theta)$；

（2） M 步：固定 $q(\boldsymbol{z}; \phi)$，寻找 θ 来最大化 $ELBO(q, \boldsymbol{x}; \theta, \phi)$.

不断重复上述两步骤，直到收敛.

在 EM 算法的每次迭代中，理论上最优的 $q(\boldsymbol{z}; \phi)$ 为隐变量的后验概率密度函数 $p(\boldsymbol{z}|\boldsymbol{x}; \theta)$，即

$$p(\boldsymbol{z}|\boldsymbol{x}; \theta) = \frac{p(\boldsymbol{x}|\boldsymbol{z}; \theta)p(\boldsymbol{z}; \theta)}{\int_{\boldsymbol{z}} p(\boldsymbol{x}|\boldsymbol{z}; \theta)p(\boldsymbol{z}; \theta)d\boldsymbol{z}}. \tag{13.5}$$

后验概率密度函数 $p(\boldsymbol{z}|\boldsymbol{x}; \theta)$ 的计算是一个统计推断问题，涉及积分计算. 当隐变量 $\boldsymbol{z}$ 是有限的一维离散变量时，计算起来比较容易. 但在一般情况下，这个后验概

率密度函数是很难计算的，通常需要通过变分推断来近似估计. 在变分推断中，为了降低复杂度，通常会选择一些比较简单的分布 $q(\boldsymbol{z};\boldsymbol{\phi})$ 来近似推断 $p(\boldsymbol{z}|\boldsymbol{x};\theta)$. 当 $p(\boldsymbol{z}|\boldsymbol{x};\theta)$ 比较复杂时，近似效果不佳. 此外，概率密度函数 $p(\boldsymbol{x}|\boldsymbol{z};\theta)$ 一般也比较复杂，很难直接用已知的分布族函数进行建模.

变分推断参见第11.4节.

变分自编码器（Variational AutoEncoder，VAE）[Kingma et al., 2014] 是一种深度生成模型，其思想是利用神经网络来分别建模两个复杂的条件概率密度函数.

（1） 用神经网络来估计变分分布 $q(\boldsymbol{z};\boldsymbol{\phi})$，称为推断网络. 理论上 $q(\boldsymbol{z};\boldsymbol{\phi})$ 可以不依赖 $\boldsymbol{x}$. 但由于 $q(\boldsymbol{z};\boldsymbol{\phi})$ 的目标是近似后验分布 $p(\boldsymbol{z}|\boldsymbol{x};\theta)$，其和 $\boldsymbol{x}$ 相关，因此变分密度函数一般写为 $q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})$. 推断网络的输入为 $\boldsymbol{x}$，输出为变分分布 $q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})$.

（2） 用神经网络来估计概率分布 $p(\boldsymbol{x}|\boldsymbol{z};\theta)$，称为生成网络. 生成网络的输入为 $\boldsymbol{z}$，输出为概率分布 $p(\boldsymbol{x}|\boldsymbol{z};\theta)$.

将推断网络和生成网络合并就得到了变分自编码器的整个网络结构，如图13.4所示，其中实线表示网络计算操作，虚线表示采样操作.

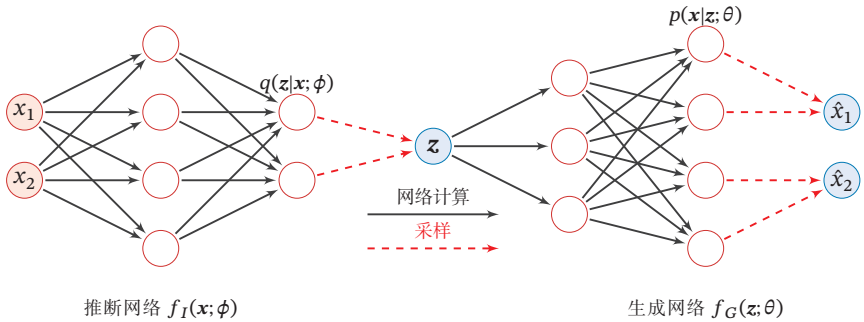


图 13.4 变分自编码器的网络结构

变分自编码器的名称来自于其整个网络结构和自编码器比较类似. 我们可以把推断网络看作“编码器”，将可观测变量映射为隐变量；把生成网络看作“解码器”，将隐变量映射为可观测变量. 然而，变分自编码器背后的原理和自编码器完全不同. 变分自编码器中的编码器和解码器的输出为分布（或分布的参数），而不是确定的编码.

自编码器参见第9.1.3节.

13.2.2 推断网络

为简单起见，假设 $q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})$ 是服从对角化协方差的高斯分布，

$$q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi}) = \mathcal{N}(\boldsymbol{z};\boldsymbol{\mu}_I, \boldsymbol{\sigma}_I^2\boldsymbol{I}), \tag{13.6}$$

其中 $\boldsymbol{\mu}_I$ 和 $\boldsymbol{\sigma}_I^2$ 是高斯分布的均值和方差，可以通过推断网络 $f_I(\boldsymbol{x};\boldsymbol{\phi})$ 来预测.

$$\begin{bmatrix}\boldsymbol{\mu}_I \\ \boldsymbol{\sigma}_I^2\end{bmatrix} = f_I(\boldsymbol{x};\boldsymbol{\phi}), \tag{13.7}$$

其中推断网络 $f_I(\boldsymbol{x};\boldsymbol{\phi})$ 可以是一般的全连接网络或卷积网络，比如一个两层的神经网络，

$$\boldsymbol{h} = \sigma\big(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\big), \tag{13.8}$$

$$\boldsymbol{\mu}_I = \boldsymbol{W}^{(2)}\boldsymbol{h} + \boldsymbol{b}^{(2)}, \tag{13.9}$$

$$\boldsymbol{\sigma}_I^2 = \text{softplus}\big(\boldsymbol{W}^{(3)}\boldsymbol{h} + \boldsymbol{b}^{(3)}\big), \tag{13.10}$$

$\text{softplus}(x) = \log(1 + e^x)$.

其中 $\boldsymbol{\phi}$ 代表所有的网络参数 $\{\boldsymbol{W}^{(1)}, \boldsymbol{W}^{(2)}, \boldsymbol{W}^{(3)}, \boldsymbol{b}^{(1)}, \boldsymbol{b}^{(2)}, \boldsymbol{b}^{(3)}\}$，$\sigma$ 和 softplus 为激活函数. 这里使用 softplus 激活函数是由于方差总是非负的. 在实际实现中，也可以用一个线性层（不需要激活函数）来预测 $\log(\boldsymbol{\sigma}_I^2)$.

推断网络的目标 推断网络的目标是使得 $q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})$ 尽可能接近真实的后验 $p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})$，需要找到一组网络参数 $\boldsymbol{\phi}^*$ 来最小化两个分布的 KL 散度，即

$$\boldsymbol{\phi}^* = \arg\min_{\boldsymbol{\phi}} \text{KL}\Big(q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi}), p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})\Big). \tag{13.11}$$

然而，直接计算上面的 KL 散度是不可能的，因为 $p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})$ 一般无法计算. 传统方法是利用采样或者变分法来近似推断. 基于采样的方法效率很低且估计也不是很准确，所以一般使用的是变分推断方法，即用简单的分布 q 去近似复杂的分布 $p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})$. 但是，在深度生成模型中，$p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})$ 通常比较复杂，很难用简单分布去近似. 因此，我们需要找到一种间接计算方法.

变分推断参见第11.4节.

根据公式(13.3)可知，变分分布 $q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})$ 与真实后验 $p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})$ 的 KL 散度等于对数边际似然 $\log p(\boldsymbol{x};\boldsymbol{\theta})$ 与其下界 $ELBO(q, \boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi})$ 的差，即

$$\text{KL}(q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi}), p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})) = \log p(\boldsymbol{x};\boldsymbol{\theta}) - ELBO(q, \boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi}), \tag{13.12}$$

因此，推断网络的目标函数可以转换为

可以看作EM算法中的E步.

$$\boldsymbol{\phi}^* = \arg\min_{\boldsymbol{\phi}} \text{KL}\Big(q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi}), p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})\Big) \tag{13.13}$$

$$= \arg\min_{\boldsymbol{\phi}} \log p(\boldsymbol{x};\boldsymbol{\theta}) - ELBO(q, \boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi}) \tag{13.14}$$

第一项与 $\boldsymbol{\phi}$ 无关.

$$= \arg\max_{\boldsymbol{\phi}} ELBO(q, \boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi}), \tag{13.15}$$

即推断网络的目标转换为寻找一组网络参数 $\boldsymbol{\phi}^*$ 使得证据下界 $ELBO(q, \boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi})$ 最大，这和变分推断中的转换类似.

参见公式(11.85).

13.2.3 生成网络

生成模型的联合分布 $p(\boldsymbol{x}, \boldsymbol{z}; \theta)$ 可以分解为两部分：隐变量 $\boldsymbol{z}$ 的先验分布 $p(\boldsymbol{z}; \theta)$ 和条件概率分布 $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$.

先验分布 $p(\boldsymbol{z}; \theta)$ 为简单起见，我们一般假设隐变量 $\boldsymbol{z}$ 的先验分布为各向同性的标准高斯分布 $\mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I})$. 隐变量 $\boldsymbol{z}$ 的每一维之间都是独立的.

条件概率分布 $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$ 条件概率分布 $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$ 可以通过生成网络来建模. 为简单起见，我们同样用参数化的分布族来表示条件概率分布 $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$，这些分布族的参数可以用生成网络计算得到.

根据变量 $\boldsymbol{x}$ 的类型不同，可以假设 $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$ 服从不同的分布族.

（1）如果 $\boldsymbol{x} \in \{0, 1\}^D$ 是 D 维的二值的向量，可以假设 $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$ 服从多变量的伯努利分布，即

$$p(\boldsymbol{x}|\boldsymbol{z}; \theta) = \prod_{d=1}^{D} p(x_d|\boldsymbol{z}; \theta) \tag{13.16}$$

$$= \prod_{d=1}^{D} \gamma_d^{x_d} (1 - \gamma_d)^{(1-x_d)}, \tag{13.17}$$

其中 $\gamma_d \triangleq p(x_d = 1|\boldsymbol{z}; \theta)$ 为第 d 维分布的参数. 分布的参数 $\boldsymbol{\gamma} = [\gamma_1, \cdots, \gamma_D]^{\mathrm{T}}$ 可以通过生成网络来预测.

（2）如果 $\boldsymbol{x} \in \mathbb{R}^D$ 是 D 维的连续向量，可以假设 $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$ 服从对角化协方差的高斯分布，即

$$p(\boldsymbol{x}|\boldsymbol{z}; \theta) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_G, \boldsymbol{\sigma}_G^2 \boldsymbol{I}), \tag{13.18}$$

其中 $\boldsymbol{\mu}_G \in \mathbb{R}^D$ 和 $\boldsymbol{\sigma}_G \in \mathbb{R}^D$ 同样可以用生成网络 $f_G(\boldsymbol{z}; \theta)$ 来预测.

生成网络的目标 生成网络 $f_G(\boldsymbol{z}; \theta)$ 的目标是找到一组网络参数 θ^* 来最大化证据下界 $ELBO(q, \boldsymbol{x}; \theta, \phi)$，即

可以看作EM算法中的M步.

$$\theta^* = \arg\max_{\theta} ELBO(q, \boldsymbol{x}; \theta, \phi). \tag{13.19}$$

13.2.4 模型汇总

结合公式(13.15)和公式(13.19)，推断网络和生成网络的目标都为最大化证据下界 $ELBO(q, \boldsymbol{x}; \theta, \phi)$. 因此，变分自编码器的总目标函数为

$$\max_{\theta, \phi} ELBO(q, \boldsymbol{x}; \theta, \phi) = \max_{\theta, \phi} \mathbb{E}_{\boldsymbol{z} \sim q(\boldsymbol{z}; \phi)} \left[\log \frac{p(\boldsymbol{x}|\boldsymbol{z}; \theta) p(\boldsymbol{z}; \theta)}{q(\boldsymbol{z}; \phi)} \right] \tag{13.20}$$

$$= \max_{\theta,\phi} \mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\phi)}\Big[\log p(\boldsymbol{x}|\boldsymbol{z};\theta)\Big] - \mathrm{KL}\Big(q(\boldsymbol{z}|\boldsymbol{x};\phi), p(\boldsymbol{z};\theta)\Big), \tag{13.21}$$

其中 $p(\boldsymbol{z};\theta)$ 为先验分布，θ 和 ϕ 分别表示生成网络和推断网络的参数.

从 EM 算法角度来看，变分自编码器优化推断网络和生成网络的过程，可以分别看作 EM 算法中的 E 步和 M 步. 但在变分自编码器中，这两步的目标合二为一，都是最大化证据下界. 此外，变分自编码器可以看作神经网络和贝叶斯网络的混合体. 贝叶斯网络中的所有节点都是随机变量. 在变分自编码器中，我们仅仅将隐藏编码对应的节点看成是随机变量，其他节点还是作为普通神经元. 这样，编码器变成一个变分推断网络，而解码器变成一个将隐变量映射到观测变量的生成网络.

我们分别来看公式(13.21)中的两项.

（1）通常情况下，公式(13.21)中第一项的期望 $\mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\phi)}[\log p(\boldsymbol{x}|\boldsymbol{z};\theta)]$ 可以通过采样的方式近似计算. 对于每个样本 $\boldsymbol{x}$，根据 $q(\boldsymbol{z}|\boldsymbol{x};\phi)$ 采集 M 个 $\boldsymbol{z}^{(m)}$，$1 \leq m \leq M$，有

$$\mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\phi)}[\log p(\boldsymbol{x}|\boldsymbol{z};\theta)] \approx \frac{1}{M}\sum_{m=1}^{M} \log p(\boldsymbol{x}|\boldsymbol{z}^{(m)};\theta). \tag{13.22}$$

期望 $\mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\phi)}[\log p(\boldsymbol{x}|\boldsymbol{z};\theta)]$ 依赖于参数 ϕ. 但在上面的近似中，这个期望变得和参数 ϕ 无关. 当使用梯度下降法来学习参数时，期望 $\mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\phi)}[\log p(\boldsymbol{x}|\boldsymbol{z};\theta)]$ 关于参数 ϕ 的梯度为 0. 这种情况是由于变量 $\boldsymbol{z}$ 和参数 ϕ 之间不是直接的确定性关系，而是一种“采样”关系. 这种情况可以通过两种方法解决：一种是再参数化，我们在下一节具体介绍；另一种是梯度估计的方法，具体参考第14.3节.

（2）公式(13.21)中第二项的 KL 散度通常可以直接计算. 特别是当 $q(\boldsymbol{z}|\boldsymbol{x};\phi)$ 和 $p(\boldsymbol{z};\theta)$ 都是正态分布时，它们的 KL 散度可以直接计算出闭式解.

给定 D 维空间中的两个正态分布 $\mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$ 和 $\mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$，其 KL 散度为

$$\begin{aligned}&\mathrm{KL}\Big(\mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1), \mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)\Big)\\ &= \frac{1}{2}\Big(\mathrm{tr}(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^{\mathrm{T}}\boldsymbol{\Sigma}_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - D + \log\frac{|\boldsymbol{\Sigma}_2|}{|\boldsymbol{\Sigma}_1|}\Big),\end{aligned} \tag{13.23}$$

其中 $\mathrm{tr}(\cdot)$ 表示矩阵的迹，$|\cdot|$ 表示矩阵的行列式.

矩阵的“迹”为主对角线（从左上方至右下方的对角线）上各个元素的总和.

这样，当 $p(\boldsymbol{z};\theta) = \mathcal{N}(\boldsymbol{z}; \boldsymbol{0}, \boldsymbol{I})$ 以及 $q(\boldsymbol{z}|\boldsymbol{x};\phi) = \mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}_I, \sigma_I^2\boldsymbol{I})$ 时，

$$\begin{aligned}&\mathrm{KL}\Big(q(\boldsymbol{z}|\boldsymbol{x};\phi), p(\boldsymbol{z};\theta)\Big)\\ &= \frac{1}{2}\Big(\mathrm{tr}(\sigma_I^2\boldsymbol{I}) + \boldsymbol{\mu}_I^{\mathrm{T}}\boldsymbol{\mu}_I - d - \log(|\sigma_I^2\boldsymbol{I}|)\Big),\end{aligned} \tag{13.24}$$

其中 $\boldsymbol{\mu}_I$ 和 σ_I 为推断网络 $f_I(\boldsymbol{x};\phi)$ 的输出.

13.2.5 再参数化

再参数化（Reparameterization）是将一个函数 $f(\theta)$ 的参数 θ 用另外一组参数表示 $\theta = g(\vartheta)$，这样函数 $f(\theta)$ 就转换成参数为 ϑ 的函数 $\hat{f}(\vartheta) = f\big(g(\vartheta)\big)$. 再参数化通常用来将原始参数转换为另外一组具有特殊属性的参数. 比如当 θ 为一个很大的矩阵时，可以使用两个低秩矩阵的乘积来再参数化，从而减少参数量.

再参数化的另一个例子是逐层归一化，参见第7.5节.

在公式(13.21)中，期望 $\mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})}\Big[\log p(\boldsymbol{x}|\boldsymbol{z};\theta)\Big]$ 依赖于分布 q 的参数 $\boldsymbol{\phi}$. 但是，由于随机变量 $\boldsymbol{z}$ 采样自后验分布 $q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})$，它们之间不是确定性关系，因此无法直接求解 $\boldsymbol{z}$ 关于参数 $\boldsymbol{\phi}$ 的导数. 这时，我们可以通过再参数化方法来将 $\boldsymbol{z}$ 和 $\boldsymbol{\phi}$ 之间随机性的采样关系转变为确定性函数关系.

我们引入一个分布为 $p(\epsilon)$ 的随机变量 ϵ，期望 $\mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})}\Big[\log p(\boldsymbol{x}|\boldsymbol{z};\theta)\Big]$ 可以重写为

$$\mathbb{E}_{\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})}\Big[\log p(\boldsymbol{x}|\boldsymbol{z};\theta)\Big] = \mathbb{E}_{\epsilon\sim p(\epsilon)}\Big[\log p(\boldsymbol{x}|g(\boldsymbol{\phi},\epsilon);\theta)\Big], \tag{13.25}$$

其中 $\boldsymbol{z} \triangleq g(\boldsymbol{\phi},\epsilon)$ 为一个确定性函数.

假设 $q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})$ 为正态分布 $N(\boldsymbol{\mu}_I, \boldsymbol{\sigma}_I^2\boldsymbol{I})$，其中 $\{\boldsymbol{\mu}_I, \sigma_I\}$ 是推断网络 $f_I(\boldsymbol{x};\boldsymbol{\phi})$ 的输出，依赖于参数 $\boldsymbol{\phi}$，我们可以通过下面方式来再参数化：

$$\boldsymbol{z} = \boldsymbol{\mu}_I + \boldsymbol{\sigma}_I \odot \epsilon, \tag{13.26}$$

参见习题13-1.

其中 $\epsilon \sim \mathcal{N}(\boldsymbol{0},\boldsymbol{I})$. 这样 $\boldsymbol{z}$ 和参数 $\boldsymbol{\phi}$ 的关系从采样关系变为确定性关系，使得 $\boldsymbol{z}\sim q(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\phi})$ 的随机性独立于参数 $\boldsymbol{\phi}$，从而可以求 $\boldsymbol{z}$ 关于 $\boldsymbol{\phi}$ 的导数.

13.2.6 训练

通过再参数化，变分自编码器可以通过梯度下降法来学习参数，从而提高变分自编码器的训练效率.

给定一个数据集 $\mathcal{D} = \{\boldsymbol{x}^{(n)}\}_{n=1}^N$，对于每个样本 $\boldsymbol{x}^{(n)}$，随机采样 M 个变量 $\epsilon^{(n,m)}, 1 \le m \le M$，并通过公式(13.26)计算 $\boldsymbol{z}^{(n,m)}$. 变分自编码器的目标函数近似为

$$\mathcal{J}(\boldsymbol{\phi},\theta|\mathcal{D}) = \sum_{n=1}^{N}\left(\frac{1}{M}\sum_{m=1}^{M}\log p(\boldsymbol{x}^{(n)}|\boldsymbol{z}^{(n,m)};\theta) - \mathrm{KL}\Big(q(\boldsymbol{z}|\boldsymbol{x}^{(n)};\boldsymbol{\phi}),\mathcal{N}(\boldsymbol{z};\boldsymbol{0},\boldsymbol{I})\Big)\right). \tag{13.27}$$

如果采用随机梯度方法，每次从数据集中采集一个样本 $\boldsymbol{x}$ 和一个对应的随机变量 ϵ，并进一步假设 $p(\boldsymbol{x}|\boldsymbol{z};\theta)$ 服从高斯分布 $\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_G,\lambda\boldsymbol{I})$，其中 $\boldsymbol{\mu}_G = f_G(\boldsymbol{z};\theta)$ 是生成网络的输出，λ 为控制方差的超参数，则目标函数可以简化为

$$\mathcal{J}(\boldsymbol{\phi},\theta|\boldsymbol{x}) = -\frac{1}{2}\|\boldsymbol{x}-\boldsymbol{\mu}_G\|^2 - \lambda\,\mathrm{KL}\Big(\mathcal{N}(\boldsymbol{\mu}_I,\boldsymbol{\sigma}_I),\mathcal{N}(\boldsymbol{0},\boldsymbol{I})\Big), \tag{13.28}$$

参见习题13-2.

其中第一项可以近似看作输入 $\boldsymbol{x}$ 的重构正确性，第二项可以看作正则化项，λ 可以看作正则化系数. 这和自编码器在形式上非常类似，但它们的内在机理是完全不同的.

参见习题13-3.

变分自编码器的训练过程如图13.5所示，其中空心矩形表示“目标函数”.

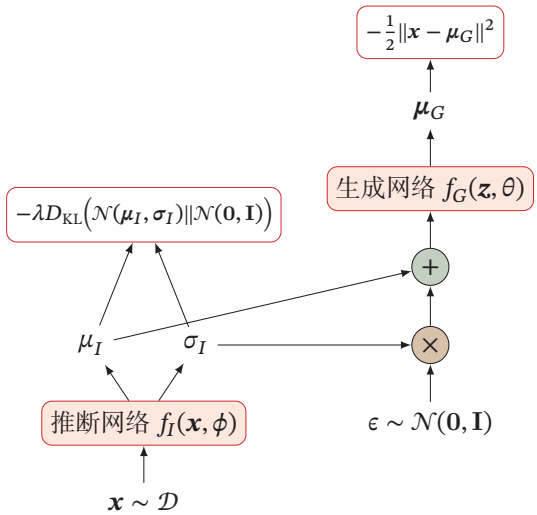
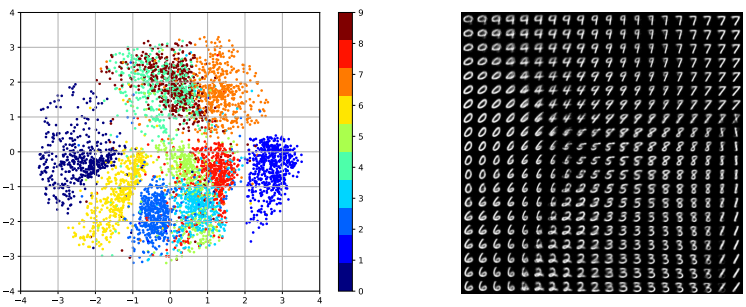


图 13.5 变分自编码器的训练过程

图13.6给出了在 MNIST 数据集上变分自编码器学习到的隐变量流形的可视化示例. 图13.6a是将训练集上每个样本 $\boldsymbol{x}$ 通过推断网络映射到 2 维的隐变量空间，图中的每个点表示 $\mathbb{E}[\boldsymbol{z}|\boldsymbol{x}]$，不同颜色表示不同的数字. 图13.6b是对 2 维的标准高斯分布上进行均匀采样得到不同的隐变量 $\boldsymbol{z}$，然后通过生成网络产生 $\mathbb{E}[\boldsymbol{x}|\boldsymbol{z}]$.



(a) 训练集上所有样本在隐空间上的投影 (b) 隐变量 $\boldsymbol{z}$ 在图像空间的投影

图 13.6 在 MNIST 数据集上变分自编码器学习到的隐变量流形的可视化示例

13.3 生成对抗网络

13.3.1 显式密度模型和隐式密度模型

之前介绍的深度生成模型，比如变分自编码器、深度信念网络等，都是显示地构建出样本的密度函数 $p(\boldsymbol{x};\theta)$，并通过最大似然估计来求解参数，称为显式密度模型（Explicit Density Model）. 比如，变分自编码器的密度函数为 $p(\boldsymbol{x},\boldsymbol{z};\theta) = p(\boldsymbol{x}|\boldsymbol{z};\theta)p(\boldsymbol{z};\theta)$. 虽然使用了神经网络来估计 $p(\boldsymbol{x}|\boldsymbol{z};\theta)$，但是我们依然假设 $p(\boldsymbol{x}|\boldsymbol{z};\theta)$ 为一个参数分布族，而神经网络只是用来预测这个参数分布族的参数. 这在某种程度上限制了神经网络的能力.

如果只是希望有一个模型能生成符合数据分布 $p_r(\boldsymbol{x})$ 的样本，那么可以不显示地估计出数据分布的密度函数. 假设在低维空间 $\mathcal{Z}$ 中有一个简单容易采样的分布 $p(\boldsymbol{z})$，$p(\boldsymbol{z})$ 通常为标准多元正态分布 $\mathcal{N}(\boldsymbol{0},\boldsymbol{I})$. 我们用神经网络构建一个映射函数 $G:\mathcal{Z}\to\mathcal{X}$，称为生成网络. 利用神经网络强大的拟合能力，使得 $G(\boldsymbol{z})$ 服从数据分布 $p_r(\boldsymbol{x})$. 这种模型就称为隐式密度模型（Implicit Density Model）. 所谓隐式模型就是指并不显式地建模 $p_r(\boldsymbol{x})$，而是建模生成过程. 图13.7给出了隐式模型生成样本的过程.

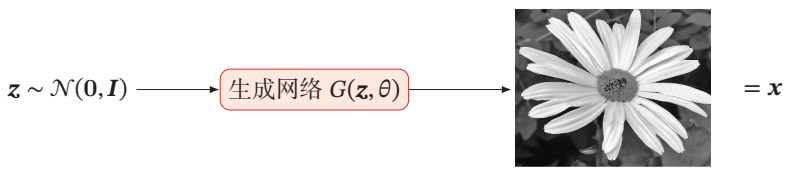


图 13.7 隐式模型生成样本的过程

13.3.2 网络分解

隐式密度模型的一个关键是如何确保生成网络产生的样本一定是服从真实的数据分布. 既然我们不构建显式密度函数，就无法通过最大似然估计等方法来训练. 生成对抗网络（Generative Adversarial Networks，GAN）[Goodfellow et al., 2014] 是通过对抗训练的方式来使得生成网络产生的样本服从真实数据分布. 在生成对抗网络中，有两个网络进行对抗训练. 一个是判别网络，目标是尽量准确地判断一个样本是来自于真实数据还是由生成网络产生；另一个是生成网络，目标是尽量生成判别网络无法区分来源的样本. 这两个目标相反的网络不断地进行交替训练. 当最后收敛时，如果判别网络再也无法判断出一个样本的来源，那么也就等价于生成网络可以生成符合真实数据分布的样本. 生成对抗网络的流程图如图13.8所示.

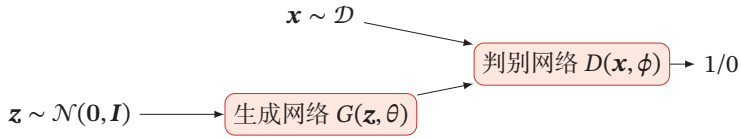


图 13.8 生成对抗网络的流程图

13.3.2.1 判别网络

判别网络（Discriminator Network）$D(\boldsymbol{x}; \phi)$ 的目标是区分出一个样本 $\boldsymbol{x}$ 是来自于真实分布 $p_r(\boldsymbol{x})$ 还是来自于生成模型 $p_\theta(\boldsymbol{x})$，因此判别网络实际上是一个二分类的分类器. 用标签 $y = 1$ 来表示样本来自真实分布，$y = 0$ 表示样本来自生成模型，判别网络 $D(\boldsymbol{x}; \phi)$ 的输出为 $\boldsymbol{x}$ 属于真实数据分布的概率，即

$$p(y = 1|\boldsymbol{x}) = D(\boldsymbol{x}; \phi), \tag{13.29}$$

则样本来自生成模型的概率为 $p(y = 0|\boldsymbol{x}) = 1 - D(\boldsymbol{x}; \phi)$.

给定一个样本 $(\boldsymbol{x}, y)$，$y = \{1, 0\}$ 表示其来自于 $p_r(\boldsymbol{x})$ 还是 $p_\theta(\boldsymbol{x})$，判别网络的目标函数为最小化交叉熵，即

$$\min_{\phi} -\Big(\mathbb{E}_{\boldsymbol{x}}\Big[y \log p(y = 1|\boldsymbol{x}) + (1 - y) \log p(y = 0|\boldsymbol{x})\Big]\Big). \tag{13.30}$$

假设分布 $p(\boldsymbol{x})$ 是由分布 $p_r(\boldsymbol{x})$ 和分布 $p_\theta(\boldsymbol{x})$ 等比例混合而成，即 $p(\boldsymbol{x}) = \frac{1}{2}\big(p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})\big)$，则上式等价于

$$\max_{\phi} \mathbb{E}_{\boldsymbol{x} \sim p_r(\boldsymbol{x})}\Big[\log D(\boldsymbol{x}; \phi)\Big] + \mathbb{E}_{\boldsymbol{x}' \sim p_\theta(\boldsymbol{x}')}\Big[\log(1 - D(\boldsymbol{x}'; \phi))\Big] \tag{13.31}$$

$$= \max_{\phi} \mathbb{E}_{\boldsymbol{x} \sim p_r(\boldsymbol{x})}\Big[\log D(\boldsymbol{x}; \phi)\Big] + \mathbb{E}_{\boldsymbol{z} \sim p(\boldsymbol{z})}\Big[\log\Big(1 - D\big(G(\boldsymbol{z}; \theta); \phi\big)\Big)\Big], \tag{13.32}$$

其中 θ 和 ϕ 分别是生成网络和判别网络的参数.

13.3.2.2 生成网络

生成网络（Generator Network）的目标刚好和判别网络相反，即让判别网络将自己生成的样本判别为真实样本.

$$\max_{\theta}\Big(\mathbb{E}_{\boldsymbol{z} \sim p(\boldsymbol{z})}\Big[\log D\Big(G(\boldsymbol{z}; \theta); \phi\Big)\Big]\Big) \tag{13.33}$$

$$= \min_{\theta}\Big(\mathbb{E}_{\boldsymbol{z} \sim p(\boldsymbol{z})}\Big[\log\Big(1 - D\big(G(\boldsymbol{z}; \theta); \phi\big)\Big)\Big]\Big). \tag{13.34}$$

上面的这两个目标函数是等价的. 但是在实际训练时，一般使用前者，因为其梯度性质更好. 我们知道，函数 $\log(x), x \in (0, 1)$ 在 x 接近 1 时的梯度要比接近 0 时的梯度小很多，接近“饱和”区间. 这样，当判别网络 D 以很高的概率认为生成网络 G 产生的样本是“假”样本，即 $\Big(1 - D\big(G(\boldsymbol{z}; \theta); \phi\big)\Big) \to 1$，这时目标函数关于 θ 的梯度反而很小，从而不利于优化.

还有一种改进生成网络的梯度的方法是将真实样本和生成样本的标签互换，即生成样本的标签为 1.

13.3.3 训练

和单目标的优化任务相比，生成对抗网络的两个网络的优化目标刚好相反. 因此生成对抗网络的训练比较难，往往不太稳定. 一般情况下，需要平衡两个网络的能力. 对于判别网络来说，一开始的判别能力不能太强，否则难以提升生成网络的能力. 但是，判别网络的判别能力也不能太弱，否则针对它训练的生成网络也不会太好. 在训练时需要使用一些技巧，使得在每次迭代中，判别网络比生成网络的能力强一些，但又不能强太多.

生成对抗网络的训练流程如算法13.1所示. 每次迭代时，判别网络更新 K 次而生成网络更新一次，即首先要保证判别网络足够强才能开始训练生成网络. 在实践中 K 是一个超参数，其取值一般取决于具体任务.

算法 13.1 生成对抗网络的训练过程

输入: 训练集 $\mathcal{D}$，对抗训练迭代次数 T，每次判别网络的训练迭代次数 K，小批量样本数量 M

随机初始化 θ, ϕ;

for $t \leftarrow 1$ **to** T **do**

  // 训练判别网络 $D(\boldsymbol{x}; \phi)$

  for $k \leftarrow 1$ **to** K **do**

    // 采集小批量训练样本

    从训练集 $\mathcal{D}$ 中采集 M 个样本 $\{\boldsymbol{x}^{(m)}\}, 1 \leq m \leq M$;

    从分布 $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ 中采集 M 个样本 $\{\boldsymbol{z}^{(m)}\}, 1 \leq m \leq M$;

    使用随机梯度上升更新 ϕ，梯度为

$$\frac{\partial}{\partial \phi}\Big[\frac{1}{M}\sum_{m=1}^{M}\Big(\log D(\boldsymbol{x}^{(m)}; \phi) + \log\big(1 - D(G(\boldsymbol{z}^{(m)}; \theta); \phi)\big)\Big)\Big];$$

  end

  // 训练生成网络 $G(\boldsymbol{z}; \theta)$

  从分布 $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ 中采集 M 个样本 $\{\boldsymbol{z}^{(m)}\}, 1 \leq m \leq M$;

  使用随机梯度上升更新 θ，梯度为

$$\frac{\partial}{\partial \theta}\Big[\frac{1}{M}\sum_{m=1}^{M} D(G(\boldsymbol{z}^{(m)}; \theta), \phi)\Big];$$

end

输出: 生成网络 $G(\boldsymbol{z}; \theta)$

13.3.4 一个生成对抗网络的具体实现：DCGAN

生成对抗网络是指一类采用对抗训练方式来进行学习的深度生成模型，其包含的判别网络和生成网络都可以根据不同的生成任务使用不同的网络结构.

本节介绍一个生成对抗网络的具体模型：深度卷积生成对抗网络（Deep Convolutional Generative Adversarial Network，DCGAN）[Radford et al., 2016]. 在 DCGAN 中，判别网络是一个传统的深度卷积网络，但使用了带步长的卷积来实现下采样操作，不用最大汇聚（pooling）操作；生成网络使用一个特殊的深度卷积网络来实现，如图13.9所示，使用微步卷积来生成 64×64 大小的图像. 第一层是全连接层，输入是从均匀分布中随机采样的100维向量 $\boldsymbol{z}$，输出是4×4×1024的向量，重塑为 $4 \times 4 \times 1024$ 的张量；然后是四层的微步卷积，没有汇聚层.

微步卷积参见第5.5.1节.

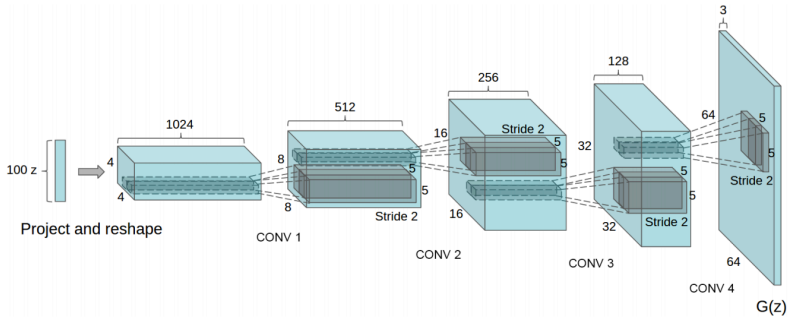


图 13.9 DCGAN 中的生成网络（图片来源：[Radford et al., 2016]）

DCGAN 的主要优点是通过一些经验性的网络结构设计使得对抗训练更加稳定. 比如：1）使用带步长的卷积（在判别网络中）和微步卷积（在生成网络中）来代替汇聚操作，以免损失信息；2）使用批量归一化；3）去除卷积层之后的全连接层；4）在生成网络中，除了最后一层使用 Tanh 激活函数外，其余层都使用 ReLU 函数；5）在判别网络中，都使用 LeakyReLU 激活函数.

13.3.5 模型分析

我们把判别网络和生成网络合并为一个整体，将整个生成对抗网络的目标函数看作最小化最大化游戏（Minimax Game）：

$$\min_{\theta}\max_{\phi}\left(\mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\left[\log D(\boldsymbol{x};\phi)\right]+\mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\left[\log\left(1-D(\boldsymbol{x};\phi)\right)\right]\right) \tag{13.35}$$

$$=\min_{\theta}\max_{\phi}\left(\mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\left[\log D(\boldsymbol{x};\phi)\right]+\mathbb{E}_{\boldsymbol{z}\sim p(\boldsymbol{z})}\left[\log\left(1-D(G(\boldsymbol{z};\theta);\phi)\right)\right]\right). \tag{13.36}$$

因为之前提到的生成网络梯度问题，这个最小化最大化形式的目标函数一般用来进行理论分析，并不是实际训练时的目标函数.

假设 $p_r(\boldsymbol{x})$ 和 $p_\theta(\boldsymbol{x})$ 已知，则最优的判别器为

参见习题13-4.

$$D^\star(\boldsymbol{x}) = \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})}. \tag{13.37}$$

将最优的判别器 $D^\star(\boldsymbol{x})$ 代入公式(13.35)，其目标函数变为

$$\begin{aligned}
\mathcal{L}(G|D^\star) &= \mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\Big[\log D^\star(\boldsymbol{x})\Big] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log(1 - D^\star(\boldsymbol{x}))\Big] && (13.38)\\
&= \mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\Big[\log \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})}\Big] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})}\Big] && (13.39)\\
&= \mathrm{KL}(p_r, p_a) + \mathrm{KL}(p_\theta, p_a) - 2\log 2 && (13.40)\\
&= 2\,\mathrm{JS}(p_r, p_\theta) - 2\log 2, && (13.41)
\end{aligned}$$

其中 $\mathrm{JS}(\cdot)$ 为 JS 散度，$p_a(\boldsymbol{x}) = \frac{1}{2}\Big(p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})\Big)$ 为一个“平均”分布.

JS 散度参见第E.3.3节.

在生成对抗网络中，当判别网络为最优时，生成网络的优化目标是最小化真实分布 p_r 和模型分布 p_θ 之间的 JS 散度. 当两个分布相同时，JS 散度为 0，最优生成网络 $G^\star$ 对应的损失为 $\mathcal{L}(G^\star|D^\star) = -2\log 2$.

13.3.5.1 训练稳定性

使用 JS 散度来训练生成对抗网络的一个问题是当两个分布没有重叠时，它们之间的 JS 散度恒等于常数 $\log 2$. 对生成网络来说，目标函数关于参数的梯度为 0，即 $\frac{\partial \mathcal{L}(G|D^\star)}{\partial \theta} = 0$.

图13.10给出了生成对抗网络中的梯度消失问题的示例. 当真实分布 p_r 和模型分布 p_θ 没有重叠时，最优的判别器 $D^\star$ 对所有生成数据的输出都为 0，即 $D^\star(G(\boldsymbol{z};\theta)) = 0, \forall \boldsymbol{z}$. 因此，生成网络的梯度消失.

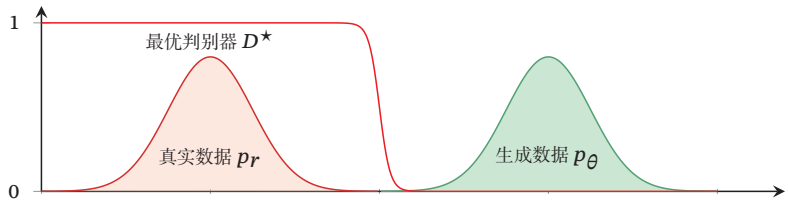


图 13.10 生成对抗网络中的梯度消失问题

因此，在实际训练生成对抗网络时，一般不会将判别网络训练到最优，只进行一步或多步梯度下降，使得生成网络的梯度依然存在. 另外，判别网络也不能太差，否则生成网络的梯度为错误的梯度. 但是，如何在梯度消失和梯度错误之间取得平衡并不是一件容易的事，这个问题使得生成对抗网络在训练时稳定性比较差.

13.3.5.2 模型坍塌

如果使用公式(13.33)作为生成网络的目标函数，将最优判别器 $D^\star$ 代入，可以得到

$$\mathcal{L}'(G|D^\star) = \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log D^\star(\boldsymbol{x})\Big] \tag{13.42}$$

$$= \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x})+p_\theta(\boldsymbol{x})}\cdot\frac{p_\theta(\boldsymbol{x})}{p_\theta(\boldsymbol{x})}\Big] \tag{13.43}$$

$$= -\mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})}\Big] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})+p_\theta(\boldsymbol{x})}\Big] \tag{13.44}$$

$$= -\mathrm{KL}(p_\theta, p_r) + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log\big(1-D^\star(\boldsymbol{x})\big)\Big] \tag{13.45}$$

$$= -\mathrm{KL}(p_\theta, p_r) + 2\,\mathrm{JS}(p_r, p_\theta) - 2\log 2 - \mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\Big[\log D^\star(\boldsymbol{x})\Big], \tag{13.46}$$

根据公式(13.41)

其中后两项和生成网络无关. 因此

$$\arg\max_\theta \mathcal{L}'(G|D^\star) = \arg\min_\theta \mathrm{KL}(p_\theta, p_r) - 2\,\mathrm{JS}(p_r, p_\theta), \tag{13.47}$$

其中 JS 散度 $\mathrm{JS}(p_\theta, p_r) \in [0, \log 2]$ 为有界函数，因此生成网络的目标更多的是受逆向 KL 散度 $\mathrm{KL}(p_\theta, p_r)$ 影响，使得生成网络更倾向于生成一些更“安全”的样本，从而造成模型坍塌（Model Collapse）问题.

前向和逆向 KL 散度 因为 KL 散度是一种非对称的散度，在计算真实分布 p_r 和模型分布 p_θ 之间的 KL 散度时，按照顺序不同，有两种 KL 散度：前向 KL 散度（Forward KL divergence）$\mathrm{KL}(p_r, p_\theta)$ 和逆向 KL 散度（Reverse KL divergence）$\mathrm{KL}(p_\theta, p_r)$. 前向和逆向 KL 散度分别定义为

$$\mathrm{KL}(p_r, p_\theta) = \int p_r(\boldsymbol{x})\log\frac{p_r(\boldsymbol{x})}{p_\theta(\boldsymbol{x})}\mathrm{d}\boldsymbol{x}, \tag{13.48}$$

$$\mathrm{KL}(p_\theta, p_r) = \int p_\theta(\boldsymbol{x})\log\frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})}\mathrm{d}\boldsymbol{x}. \tag{13.49}$$

图13.11给出数据真实分布为一个高斯混合分布，模型分布为一个单高斯分布时，使用前向和逆向 KL 散度来进行模型优化的示例. 黑色曲线为真实分布 p_r 的等高线，红色曲线为模型分布 p_θ 的等高线.

在前向 KL 散度中，

（1）当 $p_r(\boldsymbol{x}) \to 0$ 而 $p_\theta(\boldsymbol{x}) > 0$ 时，$p_r(\boldsymbol{x})\log\frac{p_r(\boldsymbol{x})}{p_\theta(\boldsymbol{x})} \to 0$. 不管 $p_\theta(\boldsymbol{x})$ 如何取值，都对前向 KL 散度的计算没有贡献.

（2）当 $p_r(\boldsymbol{x}) > 0$ 而 $p_\theta(\boldsymbol{x}) \to 0$ 时，$p_r(\boldsymbol{x})\log\frac{p_r(\boldsymbol{x})}{p_\theta(\boldsymbol{x})} \to \infty$，前向 KL 散度会变得非常大.

因此，前向 KL 散度会鼓励模型分布 $p_\theta(\boldsymbol{x})$ 尽可能覆盖所有真实分布 $p_r(\boldsymbol{x}) > 0$ 的点，而不用回避 $p_r(\boldsymbol{x}) \approx 0$ 的点.

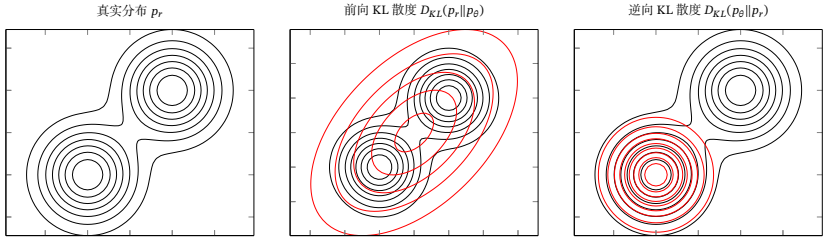


图 13.11 前向和逆向KL散度

在逆向KL散度中，

（1）当 $p_r(\boldsymbol{x}) \to 0$ 而 $p_\theta(\boldsymbol{x}) > 0$ 时，$p_\theta(\boldsymbol{x}) \log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})} \to \infty$. 即当 $p_r(\boldsymbol{x})$ 接近于0，而 $p_\theta(\boldsymbol{x})$ 有一定的密度时，逆向KL散度会变得非常大.

（2）当 $p_\theta(\boldsymbol{x}) \to 0$ 时，不管 $p_r(\boldsymbol{x})$ 如何取值，$p_\theta(\boldsymbol{x}) \log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})} \to 0$.

因此，逆向KL散度会鼓励模型分布 $p_\theta(\boldsymbol{x})$ 尽可能避开所有真实分布 $p_r(\boldsymbol{x}) \approx 0$ 的点，而不需要考虑是否覆盖所有真实分布 $p_r(\boldsymbol{x}) > 0$ 的点.

13.3.6 改进模型

在生成对抗网络中，JS散度不适合衡量生成数据分布和真实数据分布的距离. 由于通过优化交叉熵（JS散度）训练生成对抗网络会导致训练稳定性和模型坍塌问题，因此要改进生成对抗网络，就需要改变其损失函数.

13.3.6.1 W-GAN

W-GAN是一种通过用Wasserstein距离替代JS散度来优化训练的生成对抗网络 [Arjovsky et al., 2017].

Wasserstein距离也称为推土机距离，参见第E.3.4节.

对于真实分布 p_r 和模型分布 p_θ，它们的1st-Wasserstein距离为

$$\boldsymbol{W}^1(p_r, p_\theta) = \inf_{\gamma \sim \Gamma(p_r, p_\theta)} \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim\gamma}\Big[\|\boldsymbol{x} - \boldsymbol{y}\|\Big], \tag{13.50}$$

其中 $\Gamma(p_r, p_\theta)$ 是边际分布为 p_r 和 p_θ 的所有可能的联合分布集合.

当两个分布没有重叠或者重叠非常少时，它们之间的KL散度为 $+\infty$，JS散度为 $\log 2$，并不随着两个分布之间的距离而变化. 而1st-Wasserstein距离依然可以衡量两个没有重叠分布之间的距离.

两个分布 p_r 和 p_θ 的1st-Wasserstein距离通常难以直接计算，但是两个分布的1st-Wasserstein距离有一个对偶形式：

$$\boldsymbol{W}^1(p_r, p_\theta) = \sup_{\|f\|_L \le 1} \Big(\mathbb{E}_{\boldsymbol{x}\sim p_r}[f(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{x}\sim p_\theta}[f(\boldsymbol{x})]\Big), \tag{13.52}$$

数学小知识 | Lipschitz 连续函数

在数学中，对于一个实数函数 $f: \mathbb{R} \to \mathbb{R}$，如果满足函数曲线上任意两点连线的斜率一致有界，即任意两点的斜率都小于常数 $K > 0$，

$$|f(x_1) - f(x_2)| \leq K|x_1 - x_2|, \tag{13.51}$$

则函数 f 就称为K-Lipschitz 连续函数，K 称为 Lipschitz 常数.

参见习题13-5.

Lipschitz 连续要求函数在无限的区间上不能有超过线性的增长. 如果一个函数可导，并满足 Lipschitz 连续，那么导数有界. 如果一个函数可导，并且导数有界，那么函数为 Lipschitz 连续.

其中 $f: \mathbb{R}^d \to \mathbb{R}$ 为 1-Lipschitz 函数，满足

$$\|f\|_L \triangleq \sup_{x \neq y} \frac{|f(x) - f(y)|}{|x - y|} \leq 1. \tag{13.53}$$

公式(13.52)称为Kantorovich-Rubinstein 对偶定理.

根据Kantorovich-Rubinstein对偶定理，两个分布 p_r 和 p_θ 之间的 1st-Wasserstein 距离可以转换为一个满足 1-Lipschitz 连续的函数在分布 p_r 和 p_θ 下期望的差的上界. 通常情况下，1-Lipschitz 连续的约束可以宽松为 K-Lipschitz 连续. 这样分布 p_r 和 p_θ 之间的 1st-Wasserstein 距离为

$$\boldsymbol{W}^1(p_r, p_\theta) = \frac{1}{K} \sup_{\|f\|_L \leq K} \Big(\mathbb{E}_{\boldsymbol{x} \sim p_r}[f(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{x} \sim p_\theta}[f(\boldsymbol{x})] \Big). \tag{13.54}$$

参见习题13-6.

评价网络 然而，要计算公式(13.54)中的上界也并不容易. 根据神经网络的通用近似定理，我们可以假设存在一个神经网络使得可以达到这个上界. 令 $f(\boldsymbol{x}; \phi)$ 为一个神经网络，假设存在参数集合 Φ，对于所有的 $\phi \in \Phi$，$f(\boldsymbol{x}; \phi)$ 为 K-Lipschitz 连续函数，那么公式（13.54）中的上界可以近似转换为

$$\max_{\phi \in \Phi} \Big(\mathbb{E}_{x \sim p_r}[f(\boldsymbol{x}; \phi)] - \mathbb{E}_{x \sim p_\theta}[f(\boldsymbol{x}; \phi)] \Big), \tag{13.55}$$

这里忽略了常数 $\frac{1}{K}$，并不影响网络的优化.

其中 $f(\boldsymbol{x}; \phi)$ 称为评价网络（Critic Network）. 和标准 GAN 中的判别网络的值域为 $[0, 1]$ 不同，评价网络 $f(\boldsymbol{x}; \phi)$ 的最后一层为线性层，其值域没有限制. 这样只需要找到一个网络 $f(\boldsymbol{x}; \phi)$ 使其在两个分布 p_r 和 p_θ 下的期望的差最大. 即对于真实样本，$f(\boldsymbol{x}; \phi)$ 的打分要尽可能高；对于模型生成的样本，$f(\boldsymbol{x}; \phi)$ 的打分要尽可能低.

为了使得 $f(\boldsymbol{x}; \phi)$ 满足 K-Lipschitz 连续，一种近似的方法是限制参数的取值范围. 因为神经网络为连续可导函数，满足 K-Lipschitz 连续可以近似为其关于 $\boldsymbol{x}$

的偏导数的模 $\|\frac{\partial f(x;\phi)}{\partial x}\|$ 小于某个上界. 由于这个偏导数的大小一般和参数的取值范围相关，我们可以通过限制参数 ϕ 的取值范围来近似，令 $\phi \in [-c, c]$，c 为一个比较小的正数，比如 0.01.

生成网络 生成网络的目标是使得评价网络 $f(\boldsymbol{x};\phi)$ 对其生成样本的打分尽可能高，即

$$\max_{\theta} \mathbb{E}_{z\sim p(z)}\Big[f\Big(G(\boldsymbol{z};\theta);\phi\Big)\Big]. \tag{13.56}$$

因为 $f(\boldsymbol{x};\phi)$ 为不饱和函数，所以生成网络参数 θ 的梯度不会消失，理论上解决了原始 GAN 训练不稳定的问题. 并且 W-GAN 中生成网络的目标函数不再是两个分布的比率，在一定程度上缓解了模型坍塌问题，使得生成的样本具有多样性.

算法13.2给出 W-GAN 的训练过程. 和原始 GAN 相比，W-GAN 的评价网络最后一层不使用 Sigmoid 函数，损失函数不取对数.

算法 13.2 W-GAN 的训练过程

输入: 训练集 $\mathcal{D}$，对抗训练迭代次数 T，每次评价网络的训练迭代次数 K，小批量样本数量 M，参数限制大小 c;

随机初始化 θ, ϕ;

for $t \leftarrow 1$ **to** T **do**

  // 训练评价网络 $f(\boldsymbol{x};\phi)$

  for $k \leftarrow 1$ **to** K **do**

    // 采集小批量训练样本

    从训练集 $\mathcal{D}$ 中采集 M 个样本 $\{\boldsymbol{x}^{(m)}\}, 1 \le m \le M$;

    从分布 $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ 中采集 M 个样本 $\{\boldsymbol{z}^{(m)}\}, 1 \le m \le M$;

    // 计算评价网络参数 ϕ 的梯度

    $g_\phi = \frac{\partial}{\partial \phi}\Big[\frac{1}{M}\sum_{m=1}^{M}\Big(f(\boldsymbol{x}^{(m)};\phi) - f\Big(G(\boldsymbol{z}^{(m)};\theta);\phi\Big)\Big)\Big]$;

    $\phi \leftarrow \phi + \alpha \cdot \text{RMSProp}(\phi, g_\phi)$; // 使用 RMSProp 算法更新 ϕ

    $\phi \leftarrow \text{clip}(\phi, -c, c)$; // 梯度截断

  end

  // 训练生成网络 $G(\boldsymbol{z};\theta)$

  从分布 $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ 中采集 M 个样本 $\{\boldsymbol{z}^{(m)}\}, 1 \le m \le M$;

  // 更新生成网络参数 θ

  $g_\theta = \frac{\partial}{\partial \theta}\Big[\frac{1}{M}\sum_{m=1}^{M} f\Big(G(\boldsymbol{z}^{(m)};\theta);\phi\Big)\Big]$;

  $\theta \leftarrow \theta + \alpha \cdot \text{RMSProp}(\theta, g_\theta)$; // 使用 RMSProp 算法更新 θ

end

输出: 生成网络 $G(\boldsymbol{z};\theta)$

13.4 总结和深入阅读

深度生成模型是一种有机融合神经网络和概率图模型的生成模型，将神经网络作为一个概率分布的逼近器，可以拟合非常复杂的数据分布.

变分自编码器是一个非常典型的深度生成模型，利用神经网络的拟合能力来有效地解决含隐变量的概率模型中后验分布难以估计的问题 [Kingma et al., 2014; Rezende et al., 2014]. 变分自编码器的详尽介绍可以参考文献 [Doersch, 2016]. [Bowman et al., 2016] 进一步将变分自编码器应用于序列生成问题. 再参数化是变分自编码器的重要技巧. 对于离散变量的再参数化，可以使用 Gumbel-Softmax 方法 [Jang et al., 2017].

生成对抗网络 [Goodfellow et al., 2014] 是一个具有开创意义的深度生成模型，突破了以往的概率模型必须通过最大似然估计来学习参数的限制. 然而，生成对抗网络的训练通常比较困难. DCGAN[Radford et al., 2016] 是一个生成对抗网络的成功实现，可以生成十分逼真的自然图像. [Yu et al., 2017] 进一步在文本生成任务上结合生成对抗网络和强化学习来建立文本生成模型. 对抗生成网络的训练不稳定问题的一种有效解决方法是 W-GAN[Arjovsky et al., 2017]，通过用 Wasserstein 距离替代 JS 散度来进行训练.

虽然深度生成模型取得了巨大的成功，但是作为一种无监督模型，其主要的缺点是缺乏有效的客观评价，很难客观衡量不同模型之间的优劣.

习题

习题 13-1 对于一个分布为 $p_\theta(z)$ 的离散随机变量 z，以及函数 $f(z)$，如何计算期望 $\mathcal{L}(\theta) = \mathbb{E}_{z\sim p_\theta(z)}[f(z)]$ 关于分布参数 θ 的导数.

参见第13.2.5节.

习题 13-2 推导公式(13.28).

习题 13-3 通过分析公式(13.28)，给出变分自编码器和自编码器在内在机理上的不同之处.

习题 13-4 假设一个二分类问题，类别为 c_1 和 c_2，并有 $p(c_1) = p(c_2)$. 样本 $\boldsymbol{x}$ 在两个类的条件分布为 $p(\boldsymbol{x}|c_1)$ 和 $p(\boldsymbol{x}|c_2)$，一个分类器 $f(\boldsymbol{x}) = p(c_1|\boldsymbol{x})$ 用于预测一个样本 $\boldsymbol{x}$ 来自类别 c_1 的条件概率. 证明若采用交叉熵损失，

$$\mathcal{L}(f) = \mathbb{E}_{\boldsymbol{x}\sim p(\boldsymbol{x}|c_1)}\Big[\log f(\boldsymbol{x})\Big] + \mathbb{E}_{\boldsymbol{x}\sim p(\boldsymbol{x}|c_2)}\Big[\log\big(1 - f(\boldsymbol{x})\big)\Big], \tag{13.57}$$

则最优分类器 $f^\star(\boldsymbol{x})$ 为

参见公式(13.37).

$$f^\star(\boldsymbol{x}) = \frac{p(\boldsymbol{x}|c_1)}{p(\boldsymbol{x}|c_1) + p(\boldsymbol{x}|c_2)}. \tag{13.58}$$

习题 13-5 分析下面函数是否满足 Lipschitz 连续条件.

（1）$f:[-1,1]\to\mathbb{R}, f(x)=x^2$;

（2）$f:\mathbb{R}\to\mathbb{R}, f(x)=x^2$;

（3）$f:\mathbb{R}\to\mathbb{R}, f(x)=\sqrt{x^2+1}$;

（4）$f:[0,1]\to[0,1], f(x)=\sqrt{x}$.

习题 13-6 证明公式(13.54).

参考文献

Arjovsky M, Chintala S, Bottou L, 2017. Wasserstein GAN[J/OL]. CoRR, abs/1701.07875. http://arxiv.org/abs/1701.07875.

Bowman S R, Vilnis L, Vinyals O, et al., 2016. Generating sentences from a continuous space [C/OL]//Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning. 10-21. https://www.aclweb.org/anthology/K16-1002/.

Doersch C, 2016. Tutorial on variational autoencoders[J/OL]. CoRR, abs/1606.05908. http://arxiv.org/abs/1606.05908.

Goodfellow I, Pouget-Abadie J, Mirza M, et al., 2014. Generative adversarial nets[C]//Advances in Neural Information Processing Systems. 2672-2680.

Jang E, Gu S, Poole B, 2017. Categorical reparameterization with gumbel-softmax[C/OL]//Proceedings of 5th International Conference on Learning Representations. https://openreview.net/forum?id=rkE3y85ee.

Kingma D P, Welling M, 2014. Auto-encoding variational bayes[C/OL]//Proceedings of 2nd International Conference on Learning Representations. http://arxiv.org/abs/1312.6114.

Radford A, Metz L, Chintala S, 2016. Unsupervised representation learning with deep convolutional generative adversarial networks[C/OL]//Proceedings of 4th International Conference on Learning Representations. http://arxiv.org/abs/1511.06434.

Rezende D J, Mohamed S, Wierstra D, 2014. Stochastic backpropagation and approximate inference in deep generative models[J]. arXiv preprint arXiv:1401.4082.

Yu L, Zhang W, Wang J, et al., 2017. SeqGAN: Sequence generative adversarial nets with policy gradient[C]//Proceedings of Thirty-First AAAI Conference on Artificial Intelligence. 2852-2858.